

[BUG] `DepolarizingChannel` is not compatible with all interfaces

<https://github.com/PennyLaneAI/pennylane/issues/3608>

ROW 126

[BUG] DepolarizingChannel is not compatible with all interfaces #3608

[New issue](#)[Closed](#)[#3612](#)

dwierichs opened on Jan 9, 2023 · edited by dwierichs

Edits ...

Expected behavior

The channel `DepolarizingChannel` works correctly with all interfaces.

Actual behavior

1. With `torch`, the channel does not work when computing a functional jacobian, because in the creation of the Kraus matrices, `torch.tensor` objects are multiplied with simple `numpy` arrays, raising the error

```
RuntimeError: Can't call numpy() on Tensor that requires grad. Use tensor.detach().numpy() instead.
```



2. With `Tensorflow`, the channel leads to wrong results: A prefactor of `0.5` is introduced somehow.

edit

- `ResetError` and `PauliError` seem to have the same problem with `torch`.
- The bug with `Tensorflow` stems from multiplying a `tensorflow float` with a complex-valued `numpy` array, resulting in the imaginary part of `K2` being ignored. For the presented test case, the contributions of `K1` and `K2` are equal, hence the observed prefactor of `0.5`.

Additional information

No response

Source code

```
def jacobian(fn, interface=None):
    if interface=="torch":
        return partial(torch.autograd.functional.jacobian, fn)
    elif interface=="jax":
        return jax.jacobian(fn, argnums=list(range(len(inspect.signature(fn).parameters))))
    elif interface in ["tensorflow", "tf"]:
        def wrap(*args, **kwargs):
            with tf.GradientTape() as t:
                out = fn(*args, **kwargs)
            return t.jacobian(out, *args)
        return wrap
    elif interface in ["autograd", None]:
        return qml.jacobian(fn)

def array(x, interface):
    if interface=="torch":
        return torch.tensor(x, requires_grad=True)
    if interface=="jax":
        return jnp.array(x)
    if interface in ["tensorflow", "tf"]:
        return tf.Variable(x)
    if interface in ["autograd", None]:
        return pnp.array(x, requires_grad=True)

outs = []
grads = []
for interface in [None, "tensorflow", "autograd", "torch", "jax"]:
    print(interface)
    dev = qml.device("default.mixed", wires=2)
    @qml.qnode(dev, interface=interface)
    def circuit(x):
        qml.PauliX(0)
        qml.DepolarizingChannel(x, wires=[0])
        return qml.probs(wires=[0])

    x = array(0.2, interface)
    outs.append(circuit(x))
    grads.append(jacobian(circuit, interface)(x))

print(all(qml.math.allclose(o, outs[0]) for o in outs[1:]))
print(outs)
```



Assignees

No one assigned

Labels

bug 🐛

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

[Support various channels with interfaces](#)
PennyLaneAI/pennylane

Notifications

Customize

[Subscribe](#)

You're not receiving notifications from this thread.

Participants



```

print(outs)
print(all(qml.math.allclose(o, grads[0]) for o in grads[1:]))
print(grads)

```

Tracebacks

****For tensorflow:****

```

False
[tensor([0.13333333, 0.86666667], requires_grad=True), <tf.Tensor: shape=(2,), dtype=float64, numpy=array([0.13333333, 0.86666667])>]
False
[array([ 0.66666667, -0.66666667]), <tf.Tensor: shape=(2,), dtype=float32, numpy=array([ 0.66666667, -0.66666667])>]

```

****For torch:****

RuntimeError Traceback (most recent call last)

```

Cell In[20], line 13
     10     return qml.probs(wires=[0])
     12     x = array(0.2, interface)
--> 13     outs.append(circuit(x))
     14     grads.append(jacobian(circuit, interface)(x))
     16 print(all(qml.math.allclose(o, outs[0]) for o in outs[1:]))

```

File ~/repos/pennylane/pennylane/qnode.py:822, in QNode.__call__(self, *args, **kwargs)

```

819 self._tape_cached = using_custom_cache and self.tape.hash in cache
821 if qml.active_return():
--> 822     res = qml.execute(
823         [self.tape],
824         device=self.device,
825         gradient_fn=self.gradient_fn,
826         interface=self.interface,
827         gradient_kwargs=self.gradient_kwargs,
828         override_shots=override_shots,
829         **self.execute_kwargs,
830     )
832     res = res[0]
834     if old_interface == "auto":

```

File ~/repos/pennylane/pennylane/interfaces/execution.py:593, in execute(tapes, device, gradient_fn, interface)

```

492 """Execute a batch of tapes on a device in an autodifferentiable-compatible manner.
493
494 Args:

```

```

(...)
590     [ 0.          , 0.          , -0.95533649]])
591 """
592 if qml.active_return():

```

```

--> 593     return _execute_new(
594         tapes,
595         device,
596         gradient_fn,
597         interface=interface,
598         mode=mode,
599         gradient_kwargs=gradient_kwargs,
600         cache=cache,
601         cachesize=cachesize,
602         max_diff=max_diff,
603         override_shots=override_shots,
604         expand_fn=expand_fn,
605         max_expansion=max_expansion,
606         device_batch_transform=device_batch_transform,
607     )
609 if interface == "auto":
610     params = []

```

File ~/repos/pennylane/pennylane/interfaces/execution.py:388, in _execute_new(tapes, device, gradient_fn, interface)

```

384     return batch_fn(res)
386 if gradient_fn == "backprop" or interface is None:
387     return batch_fn(
--> 388         qml.interfaces.cache_execute(
389             batch_execute, cache, return_tuple=False, expand_fn=expand_fn
390         )(tapes)
391     )
393 # the default execution function is batch_execute
394 execute_fn = qml.interfaces.cache_execute(batch_execute, cache, expand_fn=expand_fn)

```

File ~/repos/pennylane/pennylane/interfaces/execution.py:206, in cache_execute.<locals>.wrapper(tapes, **kwargs)

```

202     return (res, []) if return_tuple else res
204 else:
205     # execute all unique tapes that do not exist in the cache
--> 206     res = fn(execution_tapes.values(), **kwargs)
208 final_res = []

```

```

--> 206     res = fn(execution_tapes.values(), **kwargs)
208 final_res = []
210 for i, tape in enumerate(tapes):

File ~/repos/pennylane/pennylane/interfaces/execution.py:131, in cache_execute.<locals>.fn(tapes, **kwargs)
129 def fn(tapes: Sequence[QuantumTape], **kwargs): # pylint: disable=function-redefined
130     tapes = [expand_fn(tape) for tape in tapes]
--> 131     return original_fn(tapes, **kwargs)

File /usr/lib/python3.10/contextlib.py:79, in ContextDecorator.__call__.<locals>.inner(*args, **kwargs)
76 @wraps(func)
77 def inner(*args, **kwargs):
78     with self._recreate_cm():
--> 79         return func(*args, **kwargs)

File ~/repos/pennylane/pennylane/_qubit_device.py:647, in QubitDevice.batch_execute(self, circuits)
644 # TODO: This method and the tests can be globally implemented by Device
645 # once it has the same signature in the execute() method
646 if qml.active_return():
--> 647     return self._batch_execute_new(circuits=circuits)
649 results = []
650 for circuit in circuits:
651     # we need to reset the device here, else it will
652     # not start the next computation in the zero state

File ~/repos/pennylane/pennylane/_qubit_device.py:620, in QubitDevice._batch_execute_new(self, circuits)
615 for circuit in circuits:
616     # we need to reset the device here, else it will
617     # not start the next computation in the zero state
618     self.reset()
--> 620     res = self._execute_new(circuit)
621     results.append(res)
623 if self.tracker.active:

File ~/repos/pennylane/pennylane/_qubit_device.py:375, in QubitDevice._execute_new(self, circuit, **kwargs)
372 self.check_validity(circuit.operations, circuit.observables)
374 # apply all circuit operations
--> 375 self.apply(circuit.operations, rotations=circuit.diagonalizing_gates, **kwargs)
377 # generate computational basis samples
378 if self.shots is not None:

File ~/repos/pennylane/pennylane/devices/default_mixed.py:620, in DefaultMixed.apply(self, operations, rotations)
614     raise DeviceError(
615         f"Operation {operation.name} cannot be used after other Operations have already been applied"
616         f"on a {self.short_name} device."
617     )
619 for operation in operations:
--> 620     self._apply_operation(operation)
622 # store the pre-rotated state
623 self._pre_rotated_state = self._state

File ~/repos/pennylane/pennylane/devices/default_mixed.py:542, in DefaultMixed._apply_operation(self, operation)
539     self._debugger.snapshots[len(self._debugger.snapshots)] = density_matrix
540     return
--> 542 matrices = self._get_kraus(operation)
544 if operation in diagonal_in_z_basis:
545     self._apply_diagonal_unitary(matrices, wires)

File ~/repos/pennylane/pennylane/devices/default_mixed.py:278, in DefaultMixed._get_kraus(self, operation)
275     return operation.eigvals()
277 if isinstance(operation, Channel):
--> 278     return operation.kraus_matrices()
280 return [operation.matrix()]

File ~/repos/pennylane/pennylane/operation.py:1633, in Channel.kraus_matrices(self)
1619 def kraus_matrices(self):
1620     r"""Kraus matrices of an instantiated channel
1621     in the computational basis.
1622     (...)
1631     array([[0., 0.31622777], [0., 0.]])
1632     """
--> 1633     return self.compute_kraus_matrices(*self.parameters, **self.hyperparameters)

File ~/repos/pennylane/pennylane/ops/channel.py:323, in DepolarizingChannel.compute_kraus_matrices(p)
320 if not np.is_abstract(p) and not 0.0 <= p <= 1.0:
321     raise ValueError("p must be in the interval [0,1]")
--> 323 K0 = np.sqrt(1 - p + np.eps) * np.eye(2)
324 K1 = np.sqrt(p / 3 + np.eps) * np.array([[0, 1], [1, 0]])
325 K2 = np.sqrt(p / 3 + np.eps) * np.array([[0, -1j], [1j, 0]])

File ~/venvs/dev/lib/python3.10/site-packages/torch/_tensor.py:956, in Tensor.__array__(self, dtype)
954     return handle_torch_function(Tensor.__array__, (self,), self, dtype=dtype)
955 if dtype is None:
--> 956     return self.numel()

```

```
File ~/venvs/dev/lib/python3.10/site-packages/torch/_tensor.py:956, in Tensor.__array__(self, dtype)
   954     return handle_torch_function(Tensor.__array__, (self,), self, dtype=dtype)
   955 if dtype is None:
--> 956     return self.numpy()
   957 else:
   958     return self.numpy().astype(dtype, copy=False)
```

RuntimeError: Can't call numpy() on Tensor that requires grad. Use tensor.detach().numpy() instead.

System information

Name: PennyLane
Version: 0.29.0.dev0
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: <https://github.com/XanaduAI/pennylane>
Author:
Author-email:
License: Apache License 2.0
Location: /home/david/venvs/dev/lib/python3.10/site-packages
Requires: appdirs, autograd, autoray, cachetools, networkx, numpy, pennylane-lightning, requests, retworkx, sc
Required-by: PennyLane-Lightning

Platform info: Linux-5.15.0-67-generic-x86_64-with-glibc2.35
Python version: 3.10.6
Numpy version: 1.23.5
Scipy version: 1.9.3

Installed devices:
- default.gaussian (PennyLane-0.29.0.dev0)
- default.mixed (PennyLane-0.29.0.dev0)
- default.qubit (PennyLane-0.29.0.dev0)
- default.qubit.autograd (PennyLane-0.29.0.dev0)
- default.qubit.jax (PennyLane-0.29.0.dev0)
- default.qubit.tf (PennyLane-0.29.0.dev0)
- default.qubit.torch (PennyLane-0.29.0.dev0)
- default.qutrit (PennyLane-0.29.0.dev0)
- null.qubit (PennyLane-0.29.0.dev0)
- lightning.qubit (PennyLane-Lightning-0.28.0)

Existing GitHub issues

☒ I have searched existing GitHub issues to make sure the issue does not already exist.



dwierichs added **bug** on Jan 9, 2023



dwierichs mentioned this on Jan 9, 2023

[Support various channels with interfaces #3612](#)



dwierichs closed this as **completed** in [#3612](#) on Jan 10, 2023

Support various channels with interfaces #3612

< Code

Merged dwierichs merged 8 commits into master from channels-with-interfaces on Jan 10, 2023

Conversation 11 Commits 8 Checks 0 Files changed 3 +522 -47



dwierichs commented on Jan 9, 2023 · edited Contributor

Context:
The non-unitary channels in PennyLane have some incompatibilities with the interfaces.

Description of the Change:
This PR fixes differentiability of the Kraus matrices of all channels, except for `QubitChannel` and `ThermalRelaxationChannel`. It also introduces tests for all fixed channels.

Benefits:
Differentiability support.

Possible Drawbacks:
Code complexity and performance reductions, as usual for interface support.

Related GitHub Issues:
Fixes #3608
Blocks newly added tests in #3584



dwierichs added 2 commits 3 years ago

- tests and fixes
- black

2966131
ac81e17



github-actions bot commented on Jan 9, 2023 Contributor

Hello. You may have forgotten to update the changelog!
Please edit [doc/releases/changelog-dev.md](#) with:

- A one-to-two sentence description of the change. You may include a small working example for new features.
- A link back to this PR.
- Your name (or GitHub username) in the contributors section.



Reviewers

albi3ro ✓

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

[BUG] DepolarizingChannel is not compati...

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

2 participants

